

Looking at your module overview, I can see this is a high-frequency algorithmic trading system with strict performance requirements. Before I can recommend a technology stack and development approach, I need to clarify several critical requirements that will significantly impact our architectural decisions:

I should clarify this program will be based on trading strategy to capture intra-day momentum and breakout, usually on breaking news, but not always.

Performance & Latency Requirements

1. You mentioned "delays of even a few milliseconds will have a negative impact." Can you quantify this more precisely?
 - What's the maximum acceptable latency for decision-making (e.g., <1ms, <10ms, <100ms)?
 - What's the expected throughput (trades per second, data points per second)?
 - Are we talking about high-frequency trading (microsecond latency) or lower-frequency algorithmic trading?

Decision making latency needs to be as fast as possible. From the time data is first presented from the polygon.io API, to when a buy or sell decision is made, and handed off to the Lightspeed API for execution, time is of the essence. It must be <5ms. The application will experience latency from the broker API and data module API that we cannot control, so internal latency needs to be minimized. I rely on you to make a good design recommendations to achieve this.

The trading strategy will be momentum or break-out trading. Most positions will be held for a few seconds to several minutes. The number and frequency of data points received from Lightspeed and polygon.io are unknown at this time but plan on significant amounts of data will be consumed and analyzed before and during a trade.

Trading Strategy & Risk Management

2. Position Management:

- Will this system hold multiple positions simultaneously?
- Do you need real-time portfolio risk monitoring?
- What's the expected maximum number of concurrent positions?

Multiple positions at a time are allowed but still must not exceed parameters in the rules. The Maximum number of positions should be a user defined parameter in the rules engine.

Using a risk reward ratio per-trade is not needed and can be removed from the requirements. This application will be used for trading long positions. What is needed is a max-loss per trade in terms of dollars or percent. For example, if a stock is bought for \$1.00, and the max loss is set to 10%, a stop order needs to be sent. If it's pre-market then a limit order needs to be placed. Take profit rules will also need to be defined and will be based exclusively on capturing maximum gains from price action.

3. Capital Requirements:

- What's the expected account size range (thousands, millions)?
- This affects broker API access levels and data feed costs

Dont worry about this, just track data provided

Infrastructure & Compliance

4. Production Environment:

- Will the Ubuntu server be co-located near exchanges for minimal latency?
- Do you need redundancy/failover capabilities?
- Any regulatory compliance requirements (SEC, FINRA)?

No fail over needed, no compliance requirements, and will be located near exchanges

5. Data Storage:

- How much historical data retention do you need (days, months, years)?
- Do you need tick-by-tick data storage or can you aggregate?

Actual trade data should be retained for 1 year. Tick-by-tick data needs to be retained long enough to make decisions.

It will be important to log or otherwise retain decisions made by program in conjunction with the trade data. Writing this data to disk or committing to DB is secondary to speed discussed in the performance and latency section.

Integration Constraints

6. LightSpeed Connect:

- Do you already have a LightSpeed account and API access?
- What's their API rate limiting?
- Do they provide a test/sandbox environment?

Yes, I have an account there is no rate limiting, and there is a test environment

7. Polygon.io:

- What subscription tier do you have with Polygon?

- This affects data access levels and rate limits

The tier will be stock advanced.

Once I understand these requirements, I can recommend the optimal technology stack (likely Rust or C++ for the core engine, with specific database and messaging solutions) and outline a phased development approach.

Which of these areas would you like to address first?